

Homework 6: Recommendation Systems

Big Data (BAX 423)

Instructor: Dr. Rahul Makhijani

Due: May 15, 2026 - 11:59 PM

On Canvas, only one person will submit the following files:

- A cover page with the group name, student names, and IDs
- A write-up of your responses to the Short Answer and System Design questions in all sections
- The filled-out `BAX423_HW6_Student.ipynb` notebook containing your completed code for all sections
- Screenshots and explicitly drawn plots as specified in each section's deliverables
- All prompts used if AI-assisted coding tools were used

Dataset: Food.com Recipes and Interactions

You will build a **recipe recommendation system** using the Food.com dataset, containing 180K+ recipes and 700K+ recipe reviews covering 18 years of user interactions and uploads on Food.com (formerly GeniusKitchen). This dataset was originally introduced in the following paper:

“Generating Personalized Recipes from Historical User Preferences”

Bodhisattwa Prasad Majumder, Shuyang Li, Jianmo Ni, Julian McAuley (EMNLP 2019)

Source / Download: <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>

Step 0: Generate Your Team Seed & Load Data (MANDATORY)

Every team will work with a **unique set of parameters**, generated deterministically from both team members' student IDs.

Open the `BAX423_HW6_Student.ipynb` notebook. The first section contains pre-written code for generating your team's parameters and loading the dataset. Run these cells before starting any work.

Important Note on Type-Casting: When lists are saved to a CSV, they are flattened into raw text strings. If you try to index them directly, Pandas will return the literal first character '[' instead of the actual list element. The provided code

demonstrates how to use Python's `ast.literal_eval` function to safely cast these string-encoded arrays back into natively iterable Python list objects in memory before doing any feature engineering!

Deliverables

- A screenshot of your Python notebook output showing your generated Team Assignment Parameters.

Assignment Scope: Download the two CSV files mentioned below from Kaggle and place them in the same directory as your notebook. **You will only require these two CSVs.** You are only expected to utilize the specific attributes/columns listed in these tables for all modeling and analysis tasks. Do not use any of the pre-computed or pre-processed datasets provided on Kaggle.

RAW_recipes.csv — Recipe Metadata

Field	Type	Description
name	string	Recipe title (e.g., “creamy garlic pasta”)
id	int	Unique recipe ID
minutes	int	Total preparation time in minutes
contributor_id	int	User ID of the recipe author
tags	string list	Tags: ['vegetarian', 'italian']
nutrition	float list	[Calories, Fat (PDV), Sugar (PDV), Sodium (PDV), Protein (PDV), Sat. Fat (PDV), Carbs (PDV)] <i>*(PDV = Percentage of Daily Value)*</i>
n_steps	int	Number of cooking steps
ingredients	string list	Ingredient names (e.g., ['butter', 'garlic'])
n_ingredients	int	Number of ingredients

RAW_interactions.csv — User Ratings & Reviews

Field	Type	Description
user_id	int	Unique user ID
recipe_id	int	Recipe ID (join key to RAW_recipes)
date	date	Date of the interaction
rating	int	Rating (0–5; 0 means review without a star rating)
review	string	Text of the user’s review

Note: All coding uses Python, Pandas, NumPy, scikit-learn, PyTorch, surprise, and faiss-cpu.

Section 1: Recommender Problem Framing (15 pts)

(a) Short Answer (5 pts)

- You are tasked with predicting whether a user will save a recipe. Assume you discard all “viewed but not saved” events and only train on the sparse 1-5 star ratings because they are “higher quality explicit signals.” Provide a technical argument against this proposal.
- Recommendation algorithms often process matrices where 99.99% of entries are missing. Explain why simply zero-filling these missing values (interpreting missing data as explicit “zeros” or “the user dislikes this”) is mathematically and logically disastrous for the resulting model. What structural assumption about missing data does this violate?
- There is a recommendation engine that recommends recipes based strictly on the user’s IP geolocation, current weather, and the recipe’s parsed ingredient list, without ever looking at historical user ratings. What specific recommendation taxonomy does this fall under, and why?
- Why must a mature platform like Food.com eventually abandon a pure version of the taxonomy you identified in (c) and transition to a Hybrid system?

(b) Coding (10 pts) (Fill out Notebook Section 1)

Perform the following data exploration:

- Data Cleaning:** Filter the `interactions_df` to remove all rows where `rating == 0`. For this EDA, we treat 0 as “no rating given”. Use this filtered dataset for all subsequent calculations in this section.
- Sparsity:** Compute the number of unique users and recipes in your filtered dataset. Compute sparsity:

$$\text{Sparsity} = 1 - \frac{\text{Observed Ratings (total filtered rows)}}{\text{Users} \times \text{Recipes}}$$

- c. Rating Distribution:** Use `value_counts().sort_index()` to count the frequencies of ratings 1–5. Plot these as a bar chart. Is it symmetric or skewed? How does this mathematically illustrate the concept of selection bias in explicit feedback datasets?
- d. Power Users:** Calculate user frequencies. How many users have `> MIN_REVIEWS_PER_USER` ratings? What percentage of total ratings come from the top `TOP_N` users?
- e. Most Popular vs. Best Rated:** Use `groupby('recipe_id').agg()` to compute total ratings and average rating per recipe. Create two Top-N lists: one ranked by total count (popularity) and one ranked by average star rating (quality). For the quality list, explicitly filter to only include recipes with `≥ MIN_REVIEWS_PER_ITEM` ratings. Merge with recipe names and compare the two lists.

Deliverables

- The filled-out EDA section within your `BAX423_HW6_Student.ipynb`.
- A drawn plot/screenshot of the bar chart showing the rating distribution.
- A short written explanation for part (b) describing the skewness of the ratings and the mathematical implication for selection bias.
- A short written sentence for part (d) answering if both lists match and your conclusion regarding popularity bias.

Section 2: Collaborative Filtering & Latent Factor Models (20 pts)

(a) Short Answer (3 pts)

- a.** In an item-based neighborhood CF model for food recipes, we precompute a dense $N \times N$ similarity matrix. If Food.com scales from 100,000 recipes to 10,000,000 recipes, what specific hardware bottlenecks (computation and memory) will this approach hit? Express your answer using Big-O time and space complexity scaling.
- b.** A developer decides to use Pearson Correlation instead of Jaccard Similarity to compare users based on their recipe history. Under what specific conditions in the user interaction data would Pearson be mathematically undefined or produce NaN values, while Jaccard would still compute perfectly?

(b) Coding (17 pts) (Fill out Notebook Section 2)

Build a collaborative filtering model using the `surprise` library.

- a. Data Prep & Filtering:** Filter out `rating == 0`. Use `value_counts()` to keep only users with `≥ MIN_REVIEWS_PER_USER` ratings and recipes with `≥ MIN_REVIEWS_PER_ITEM`

ratings. **Print** the DataFrame shape before and after filtering, and the number of unique users/recipes remaining.

- b. Train/Test Split:** Load the filtered data into a Surprise `Dataset` using `Reader(rating_scale=(1, 5))`. Split using `train_test_split` with your `TEST_SPLIT` and `RANDOM_SEED`.
- c. Train SVD:** Train `surprise.SVD` with `n_factors=LATENT_FACTORS` and `n_epochs=20`. Evaluate RMSE on the test set using `accuracy.rmse()`. **Print** the RMSE.
- d. Baseline:** Train `surprise.BaselineOnly()` on the same data. **Print both RMSE values side-by-side** and the improvement. Write 1–2 sentences comparing SVD to the baseline.
- e. Hyperparameter Experiment:** Train a second SVD with `n_factors = LATENT_FACTORS × 3`. **Print all three RMSE values** (Baseline, SVD, $3\times$ SVD). Write 2–3 sentences explaining whether more factors helped and why.
- f. Top-N Recommendations:** Find the most prolific user. Identify all `recipe_ids` they have **not** rated. Use `model.predict()` to estimate ratings, sort descending, take the top `TOP_N`, and merge with `recipes_df`. **Print** a table showing: recipe name, ingredients, and predicted rating.

Deliverables

- The filled-out Collaborative Filtering section within your **BAX423.HW6_Student.ipynb**.
- A screenshot of the **printed output** showing DataFrame shape before/after filtering and unique user/recipe counts.
- A screenshot of the **printed RMSE comparison** showing all three values: Baseline, SVD, and $3\times$ SVD.
- A written 2–3 sentence explanation analyzing whether more latent factors improved performance and why.
- A screenshot of the **printed Top-N recommendation table** with recipe name, ingredients, and predicted rating.

Section 3: Two-Tower Models & Candidate Generation (25 pts)

(a) Short Answer (3 pts)

- a.** You are designing a Two-Tower model. An engineer proposes passing the raw “Recipe ID” directly into the User Tower, since the user interacted with that recipe in the past.

Explain the strict architectural rule this violates regarding decoupled caching, and how you must restructure the input to keep serving latency under 50ms.

(b) Coding (22 pts)

You will implement a two-tower retrieval model in PyTorch and use the `faiss` library to build a high-performance approximate nearest neighbor (ANN) index for candidate retrieval.

a. Feature Engineering: (Fill out Notebook Section 3.1)

- **Binary Labels:** Create a target column where `rating ≥ 4` is `1` (positive) and all else is `0`.
- **Historical Stats:** Group your dataset by User and Recipe to calculate 4 historical statistics: `user_review_count`, `user_avg_rating`, `recipe_review_count`, and `recipe_avg_rating`.
- **Recipe Metadata:** Merge your dataframe with the recipe metadata to include 4 continuous features: `n_ingredients`, `minutes`, `calories`, and `n_steps`.
- **Imputation & Clipping:** Fill any missing `NaN` values with the median. Clip extreme outliers by capping `minutes` at a maximum of 300 and `calories` at 2000.

b. Encoding & Normalization: (Fill out Notebook Section 3.2)

- **Label Encoding:** Neural network embedding tables require contiguous, 0-indexed integers. You **must** use `sklearn.preprocessing.LabelEncoder` to transform the raw `user_id` and `recipe_id` into integer `user_idx` and `item_idx` columns.
- **Standardization:** Use `sklearn.preprocessing.StandardScaler` to normalize all 8 of the continuous numerical columns generated in the steps above.

c. Implement Two-Tower Model: (Fill out Notebook Section 3.3) Implement `UserTower` and `ItemTower` within the `TwoTowerModel` architecture. For each tower:

- Initialize a PyTorch `nn.Embedding` layer that projects the raw user/item ID into `EMBEDDING_DIM`.
- In the forward pass, concatenate this ID embedding with the respective dense continuous features.
- Pass the concatenated vector through an `nn.Sequential` MLP configured to project the tensor down to the final shared embedding space:
 1. `nn.Linear(in_features, out_features = 128)`

— For the **User Tower**, `in_features = EMBEDDING_DIM + len(user_cont_cols)`
(where `user_cont_cols` is `['user_review_count', 'user_avg_rating']`).

— For the **Item Tower**, `in_features = EMBEDDING_DIM + len(item_cont_cols)`
(where `item_cont_cols` is `['recipe_review_count', 'recipe_avg_rating', 'n_ingredients', 'minutes', 'calories', 'n_steps']`).

2. `nn.ReLU()`

3. `nn.Linear(in_features = 128, out_features = EMBEDDING_DIM)`

- Apply `F.normalize(p=2, dim=1)` to the final output of both towers before computing their dot product in the main `TwoTowerModel` forward pass.

d. Dataset & DataLoaders: (Fill out Notebook Section 3.4) Construct the `RecipeDataset` class and its batched iterables:

- `__init__`: Extract and store the necessary PyTorch tensors from the dataframe (user IDs, item IDs, user dense features, item dense features, and binary labels).
- `__len__`: Return the total number of samples in the dataset.
- `__getitem__`: Return a tuple of exactly five elements for a given index: `(user_id, user_feat, item_id, item_feat, label)`.
- **Splitting**: Split your main dataframe into train and test sets using the provided `TRAIN_SPLIT` threshold.
- **DataLoaders**: Instantiate PyTorch `DataLoader` objects for both sets, setting `batch_size=1024`.

e. Train: (Fill out Notebook Section 3.5) Initialize an optimizer using `torch.optim.Adam` (`lr=0.001`) and the `nn.BCEWithLogitsLoss()` criterion. Inside the provided training loop, implement the standard PyTorch sequence for each batch:

- Zero the optimizer gradients.
- Pass the user and item tensors through the model to compute the forward prediction scores.
- Calculate the loss between the prediction scores and the true `label.float()`.
- Backpropagate the error (`loss.backward()`).
- Step the optimizer to update the model weights.

f. Evaluate: (Fill out Notebook Section 3.6) Quantitatively evaluate your retrieval model's performance on the test set:

- **evaluate_hit_at_k:** For each test user, compute the dot product between their embedding and the `all_item_embs` matrix. Sort the resulting scores in descending order. Check if the user's actual positive test items fall within the top 10 or 50 predicted indices, and calculate the overall `Hit@10` and `Hit@50` ratios.
- **Extract Item Embeddings:** Extract all unique items from your dataframe and pass them through your trained `item_tower` to generate the unified `all_item_embs` reference array.
- **Run Evaluation:** Call your evaluation function using the test dataframe and print out the final `Hit@10` and `Hit@50` metric scores.

g. FAISS ANN Index: (Fill out Notebook Section 3.7) Construct the high-performance candidate retrieval index using the Facebook AI Similarity Search (`faiss`) library:

- **Prepare Embeddings:** Extract your user query and item embeddings as NumPy `float32` arrays, and explicitly normalize them using `faiss.normalize_L2` so that Inner Product search equates to Cosine Similarity.
- **Exact Search:** Create a brute-force `faiss.IndexFlatIP` index to establish the ground truth nearest neighbors.
- **Approximate Search:** Create a partitioned `faiss.IndexIVFFlat` index using `nlist=100`. Train the quantizer on the item embeddings, and then add the items to the index.
- **Recall vs. Latency Sweep:** Loop over `nprobe_values = [1, 2, 5, 10, 20, 50, 100]`. For each `nprobe`, execute the search, calculate the latency, calculate the Recall percentage against the exact ground truth, and print the tradeoff results to the console.
- **Plot the Tradeoff:** Use `matplotlib` to generate a line plot of the Recall vs. Latency tradeoff. Place Latency (ms) on the x-axis and Recall on the y-axis, and visually annotate each data point with its corresponding `nprobe` value.

Deliverables

- The filled-out Two-Tower and FAISS sections within your **BAX423_HW6_Student.ipynb**.
- Screenshots showing the exact numerical output of your `Hit@10` and `Hit@50` scores.
- Screenshots showing your FAISS console output for Recall and Latency at each

nprobe value.

- A generated plot screenshot showing the FAISS IVF Recall vs. Latency tradeoff.

Section 4: The Ranking Stage & Deep Cross Networks (25 pts)

(a) Short Answer (3 pts)

- a. You have optimized the final Ranker purely for CTR (Click-Through Rate). As a result, the app began recommending sensationalized, low-quality recipes to maximize clicks (“clickbait”). Users clicked, but immediately abandoned their carts, crashing platform revenue. Design a multi-objective loss function or calibration layer that statistically balances at most two objectives (for example, choose two from: CTR, Cart-Conversion Rate, or Actual Time Spent Cooking).

(b) Coding (22 pts)

Build a Deep & Cross Network (DCN v2) to rank the candidate recipes.

- a. **Data Prep:** (Fill out Notebook Section 4.1) Prepare the final dataframe (`model_df_dcn`) for the ranking stage:
- **Merge:** Construct `model_df_dcn` by merging the raw interaction data with all 8 historical user and recipe features calculated in Section 3.
 - **Labels:** Transform the raw ratings into binary labels (`1` if ≥ 4 , else `0`).
 - **Scaling:** Extract the combined 8 continuous dense features and scale them using `sklearn.preprocessing.StandardScaler` .
 - **Encoding:** Use `sklearn.preprocessing.LabelEncoder` to map the raw user and recipe IDs into contiguous integer indices.
 - **Global Variables:** Calculate and store the `num_users_dcn` , `num_items_dcn` , and `dense_dim` integers to initialize the PyTorch architecture in the next step.
- b. **Implement DCN v2 & Baseline:** (Fill out Notebook Section 4.2) Construct the Deep & Cross Network architecture and a baseline DNN. You must implement three PyTorch classes:
- **CrossLayer :** Implement the explicit cross layer mathematically defined as $x_{l+1} = x_0 \circ (W_l x_l + b_l) + x_l$.
 - **DCNv2 :** Construct the full architecture:
 - `nn.Embedding(num_users, 16)` and `nn.Embedding(num_items, 16)`

- Concatenate `[user_emb, item_emb, dense_feats]` $\rightarrow x_0$ of dimension $16 + 16 + 8 = 40$ (where `dense_feats` is composed of the 8 continuous columns: `['user_review_count', 'user_avg_rating', 'n_ingredients', 'minutes', 'calories', 'n_steps', 'recipe_avg_rating', 'recipe_review_count']`).
 - **Cross Network:** Stack N `CrossLayer` modules. (In PyTorch, each layer must take both x_0 and the current state x_l).
 - **Deep Network:** Stack layers dynamically using the `deep_layers=[256, 128, 64]` argument:
`Linear(40 $\rightarrow$ 256)  $\rightarrow$  ReLU  $\rightarrow$  Batch Normalization (BN)  $\rightarrow$`
`Linear(256 $\rightarrow$ 128)  $\rightarrow$  ReLU  $\rightarrow$  Batch Normalization (BN)  $\rightarrow$`
`Linear(128 $\rightarrow$ 64)  $\rightarrow$  ReLU  $\rightarrow$  Batch Normalization (BN)`
 - **Combine:** Concatenate cross output (dim 40) + deep output (dim 64) $\rightarrow$ `Linear(104 $\rightarrow$ 1)  $\rightarrow$  Sigmoid`
- c. Dataset & DataLoaders:** (Fill out Notebook Section 4.3) Construct the `DCNRankerDataset` class and its batched iterables:
- `__init__`: Extract and store the necessary PyTorch tensors from the dataframe (`user_idx`, `item_idx`, `dense_feats`, and `label`).
 - `__len__`: Return the total number of samples in the dataset.
 - `__getitem__`: Return a tuple of exactly four elements for a given index: (`user_idx`, `item_idx`, `dense_feats`, `label`).
 - **DataLoaders:** Instantiate the `DCNRankerDataset` for both the train and test splits, and wrap them in PyTorch `DataLoader` iterables (set `batch_size=1024`).
- d. Train & Evaluate:** (Fill out Notebook Section 4.4) Train both models and compare their classification performance:
- **Initialize Models:** Instantiate the `DCNv2` and `PlainDNN` architectures and move them to the device. Define the `nn.BCEWithLogitsLoss` criterion.
 - **Training Loop:** Implement the `train_ranker` function. For each epoch, run the standard PyTorch training sequence. At the end of training, evaluate the model on

the `test_loader` to calculate and return the final `roc_auc_score` and `log_loss`.

- **Execution:** Call your `train_ranker` loop on both the `DCNv2` and the `PlainDNN` for 5 epochs.
 - **Comparison:** Print the resulting Test AUC and Log-Loss metrics for both models, and generate a grouped bar chart visually comparing their performance.
- e. **Ablation Study:** (Fill out Notebook Section 4.5) Determine if stacking deeper cross-networks improves recommendation accuracy on this dataset:
- **Sweep:** Loop through $N \in [1, 2, 4]$. For each N , instantiate a new `DCNv2` model with exactly N crossed layers, train it using `train_ranker`, and append the resulting `(N, Test AUC, Test Log-Loss)` tuple to the `ablation_results` list.
 - **Print:** The provided formatting code will print a table comparing the performance of all 3 DCN configurations against your baseline Plain DNN.
 - **Conclusion:** Provide a 2-sentence written conclusion identifying which configuration performed best and explaining whether deeper cross-networks improved performance.

Deliverables

- The filled-out DCN v2 and Ablation Study sections within your **BAX423_HW6_Student.ipynb**.
- The generated grouped bar chart comparing the Test AUC and Log-Loss of the Plain DNN vs. DCN v2.
- A screenshot of your ablation study results (1, 2, and 4 layers) and a written conclusion identifying which depth was most effective.

Section 5: Case Study: YouTube Recommendations (15 pts)

(a) Short Answer

Read the seminal paper *Deep Neural Networks for YouTube Recommendations* (Covington et al., 2016). **Link:** <https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf>

- a. The paper states that users strongly prefer fresh content, but machine learning models naturally exhibit an implicit bias toward older videos because older videos have accumulated far more watch time and clicks. How exactly did the YouTube engineering team manipulate the input features to effectively teach the model to capture this “freshness” bias?
- b. YouTube’s final ranking model could be trained to predict simple click-through-rate

(CTR), but they explicitly abandoned this approach. What specific metric did they optimize instead, and how did they adjust the weighted logistic regression loss function during training to mathematically approximate this new metric?